

Decoding Italian Music: A Text Mining Breakdown via Topic Modeling and Classification

Mattia Bertolotti (866362), Gabriele Giorgio (925678)

15 June 2026

Text Mining and Search A.Y. 2025/2026

CdLM in Theory and Technology of Communication

Università degli Studi di Milano-Bicocca

Abstract

This project investigates the linguistic and thematic patterns within over 20,000 Italian song lyrics. The study begins with a complex text pre-processing phase, a particularly challenging task due to the specificities of the Italian language. Subsequently, topic modeling is applied with the dual objective of comparing the performance of different algorithms and identifying the dominant themes in the national music scene. The analysis continues with text classification techniques, aimed at verifying the lexical consistency between an artist's lyrics and their musical genre. Finally, the project concludes with a specific thematic focus, analyzing how Italian artists approach and express the theme of love.

Indice

1	Introduction	3
2	Dataset creation	3
3	Text Pre-Processing and Text Representation	4
3.1	Language Detection	4
3.2	Tokenization, Normalization, Lemmatization	5
3.3	Text Representation	7
4	Topic modeling	8
4.1	LDA, NMF and LSA	8
4.1.1	Topic Extraction and Interpretation: Implementing NMF	10
4.2	Neural Topic Modeling: Building a VAE with PyTorch	11
4.3	Comparative Analysis: NMF vs. Neural VAE	13
5	Text Classification	15
5.1	Genre Pre-Processing	16
5.2	Machine Learning	17
5.2.1	Support Vector Machines via One-vs-Rest	18
5.2.2	Linear Classifier Chains	19
5.2.3	Tree-Based Classifier Chains (XGBoost)	20
5.3	Transformer BERT	21
5.4	Comparative Analysis: ML vs. BERT	24
5.5	Sentiment Analysis	26
6	Conclusion	27
	Bibliography	29

1 Introduction

Please note that the drafting of this report was supported by a Large Language Model (Gemini). The artificial intelligence was employed exclusively as a linguistic aid to refine the vocabulary, improve readability, and perform stylistic revisions, thereby ensuring a standard of presentation appropriate for an academic context.

For the purposes of this report, the source code has been omitted in favor of an in-depth description of the applied methodologies and a discussion of the results. However, to ensure full transparency and research reproducibility, the entire computational framework has been made available in a dedicated GitHub repository. Within the repository, the code is organized into interactive notebooks, where each logical block is accompanied by explanatory comments.

Project repository

The complete code and analysis procedures are available on GitHub.
github.com/gabriele11231/Text_mining_italian_artists_lyrics

The music industry is constantly enriched by new releases, predominantly dropping on traditional ‘New Music Fridays’. This project focuses specifically on the Italian music scene, an ecosystem characterized by a vast heterogeneity of genres ranging from the classic rock of Vasco Rossi and the indie pop of Calcutta, to the current Hip-Hop landscape.

The primary objective of this research was to evaluate the effectiveness of **text pre-processing** techniques applied to Italian lyrics, specifically tackling the complex challenge of lemmatization, a well-known critical pain point for this language. Subsequently, through **topic modeling** algorithms, the main themes embedded within the songs were identified. The investigation then extended to **text classification**, aiming to verify the degree of consistency between the artists’ lexical choices and the genre tags assigned to them by users via the [Last.fm](https://last.fm) platform.

Finally, the analysis concludes with a look at how Italian artists approach and articulate the theme of love, a narrative thread that has historically played a central role in our musical tradition.

2 Dataset creation

Regarding the construction of the dataset, the starting point was a CSV file containing over 4,000 artist names. Data acquisition was performed via the Spotify API, prior to the implementation of their current access restrictions. The data collection strategy employed a snowball sampling approach: using the artist ‘Calcutta’ as the initial seed node, the list of his ‘related artists’ was extracted. This process was then applied recursively to each newly discovered artist, progressively expanding the network until the comprehensive list was compiled.

Artists dataset from Spotify:

id The artist’s unique identifier provided by Spotify.

name The artist’s name.

genres The artist’s genres provided by Spotify.

popularity A Spotify metric indicating how popular the artist is.

followers The number of followers an artist has on Spotify.

image-url A link to the artist’s profile picture.

From this initial dataset, only artists with a popularity index of 25 or higher were retained. This threshold was established to exclude musical projects with marginal consumer reach. Subsequently, using the artist’s name, via the Last.fm API, the top five user-voted genres, along with the total number of listeners and playcounts, were retrieved for each artist. We did not use Spotify’s provided genres because the platform is currently changing how it assigns genres to artists, and many of them no longer have one. Finally, the textual corpus was constructed by extracting the 20 most popular tracks per artist from the [Genius platform](#), yielding a total volume of over 25,000 songs.

The data acquisition process required several days of computation due to the strict rate limits imposed by the Genius API. To circumvent this obstacle, an API key rotation system was implemented, automatically switching to an alternate key when the current one exceeded its quota. Furthermore, the extraction pipeline was equipped with a checkpointing mechanism, designed to resume the download from the exact point of interruption in the event of a failure. The complete resulting dataset was ultimately serialized and stored in a JSON file.

Artists lyrics dataset:

artist_id The artist’s unique identifier provided by Spotify.

artist_name The artist’s name.

genres The artist’s top 5 genres provided by Last.fm.

listeners Number of listeners on Last.fm.

playcount Number of playcount on last.fm.

tracks List of artist’s tracks containing title and lyrics.

3 Text Pre-Processing and Text Representation

The first phase of the text mining pipeline involves data ingestion and an initial exploratory data analysis of the corpus. By sampling the first record, specifically extracting the artist’s identifier, name, track title, and a truncated preview of the raw text, we verify the integrity of the loaded corpus. This exploratory step is crucial to establish a baseline understanding of the raw text’s condition and formatting before applying any Natural Language Processing (NLP) pre-processing techniques.

3.1 Language Detection

The initial steps for cleaning text depend heavily on correctly identifying the language. Musical lyrics, however, make this difficult because they often mix multiple languages together a phenomenon known as ‘code-switching’. For example, a single song might blend Italian with English or Spanish in the same verse. Additionally, some tracks might be entirely in a foreign language. Because of this, using a standard tool that assigns only one language to a whole song isn’t effective. It could either leave unwanted foreign words in the data, or accidentally delete a mostly Italian song just because it has a foreign chorus.

To address this complexity, a granular, line-by-line language detection strategy was implemented. The process leverages the python **Lingua** ([Stahl, 2021](#)) library, explicitly configured with a predefined set of expected languages (Italian, English, Spanish, French, Russian, German, and Portuguese). Providing

this context bounds the search space of the algorithm, significantly improve detection accuracy and reduce false positives. The filtering pipeline operates in two distinct phases:

- **Line-Level Language Filtering:** The raw lyrics are segmented by newline characters. Each non-empty line is evaluated by the language detector. Only lines strictly classified as Italian are retained, while foreign or unrecognized lines are systematically pruned from the text.
- **Document-Level Length Pruning:** After the removal of foreign lines, the remaining Italian text is re-evaluated for length and substance. Any track retaining fewer than 8 valid Italian lines is entirely discarded. This step is crucial because extremely short texts lack the sufficient statistical properties (such as term frequency) required for robust text representation and downstream NLP tasks.

After this cleanup, the remaining Italian lyrics are ready to be processed.

3.2 Tokenization, Normalization, Lemmatization

Before committing to the final normalization pipeline, an initial experimental phase was conducted using **spaCy** (Honnibal et al., 2020), widely regarded as an industry standard for Natural Language Processing tasks. As a first step, to reduce the computational overhead on the neural NLP models, a fast heuristic cleaning function was implemented using **Regular Expressions** (re). This function executes prior to any deep linguistic analysis. It replaces newline characters with spaces and systematically removes all non-alphabetic characters. Crucially, Italian accented vowels (à, è, é, ì, í, ò, ó, ù, ú) were explicitly preserved, as diacritics are essential for lexical disambiguation in the Italian language (e.g., distinguishing *pero* from *però*). Finally, the text was converted to lowercase and stripped of redundant whitespaces. For the core NLP tasks, tokenization and lemmatization, we selected spaCy’s large Italian model (`it_core_news_lg`) to maximize theoretical accuracy. To handle the volume of the corpus efficiently, the nested dictionary structure was temporarily flattened into a single list of raw strings. This allowed the utilization of `nlp.pipe()`, leveraging multiprocessing (`n_process=-1`) and batching (`batch_size=100`) to distribute the workload across all available CPU cores. During this process, tokens shorter than two characters were also filtered out to remove residual noise.

Despite the high computational efficiency achieved through multiprocessing, a qualitative evaluation of the resulting corpus revealed critical inaccuracies in spaCy’s lemmatization process for the Italian language. When testing the pipeline on the track “Tutti” by the artist Calcutta, we discovered severe morphological mapping errors. Most notably, the conjugated verb “**vesto**” (first-person singular of *vestire*, meaning “I dress”) was incorrectly lemmatized to “**vestare**”, a completely non-existent word in the Italian lexicon. Because lemmatization is the foundational step for reducing vocabulary dimensionality, generating pseudo-words or incorrect lemmas introduces unacceptable noise into the dataset, corrupting term frequencies and downstream models. Due to these critical qualitative failures, the spaCy implementation was ultimately abandoned. This evaluation directly motivated the transition to **Stanza** (Qi et al., 2020), which, despite being computationally heavier, provides the morphological precision required for a robust text mining project.

Relying on deep neural networks, Stanza (developed by the Stanford University) provides exceptionally robust native support for the Italian language and is explicitly designed to leverage hardware acceleration via **PyTorch** (Paszke et al., 2019). Also here, before feeding the corpus into the computationally expensive neural model, a fast heuristic text normalization step is applied using Regular Expressions (re). Stanza operates on a strict pipeline architecture. Deep neural networks cannot extract a word’s dictionary root directly from raw text in a single pass; rather, the linguistic

analysis must be executed sequentially. Each processor in the pipeline resolves specific linguistic features and structures the data for the subsequent module. To optimize RAM usage and processing time, the pipeline is initialized strictly with the required components: `stanza.Pipeline(lang='it', processors='tokenize,pos,lemma', use_gpu=True)`.

The applied processors function as follows:

- **Tokenization (tokenize)**: This foundational step segments the raw, flat strings into discrete units. It first performs Sentence Segmentation, understanding sentence boundaries despite the lack of traditional punctuation often found in song lyrics, and subsequently applies Tokenization, splitting each sentence into individual words.
- **Part-Of-Speech Tagging (pos)**: The POS tagger examines each token within the syntactic context of its surrounding sentence to assign a definitive grammatical label (Noun, Verb, Adjective, Adverb). This contextual understanding drastically improves accuracy over traditional heuristic models.
- **Context-Aware Lemmatization (lemma)**: Operating on the output of the previous steps, the lemmatizer takes the original token alongside its POS tag to accurately compute the dictionary root of the word (the lemma). Thanks to the contextual disambiguation provided by the POS tagger, the neural lemmatizer operates flawlessly, effectively resolving the morphological ambiguities that caused previous models to fail on conjugated Italian verbs.

The integration of Stanza’s POS tagger enabled the implementation of a highly sophisticated, grammar-based filtering mechanism. Standard pre-processing pipelines often rely on static “stop-word” dictionaries to remove semantically empty terms (like articles, conjunctions, and prepositions). However, static lists are rigid and often context-blind. Instead, our pipeline dynamically filters the text by evaluating the assigned syntactic role of each token.

A strict inclusion criteria (**VALID_POS_TAGS**) was defined to retain only words carrying heavy semantic weight: **Nouns (NOUN)**, **Verbs (VERB)**, **Adjectives (ADJ)**, and **Adverbs (ADV)**. All other grammatical classes are systematically discarded. Coupled with a secondary filter that removes excessively short tokens (length < 2), this grammatical pruning acts as a highly advanced stop-word remover. Extracting pure, semantically dense lemmas is the ultimate goal of this phase, as it drastically reduces the vocabulary dimensionality and eliminates structural noise from the subsequent Bag-of-Words (BoW) and TF-IDF mathematical matrices.

It is critical to note that while deep learning models yield superior quality, they introduce a substantial computational burden. Despite actively utilizing GPU acceleration to process tensors in batches, Stanza’s neural pipeline is inherently slower than heuristic or statistical models like spaCy. On our specific hardware configuration, which includes a high-end **CPU (Intel Ultra 7 265k)**, **32GB of DDR5 6400MHz RAM**, and a **dedicated GPU (Nvidia RTX 5070 FE)**, the complete neural processing and lemmatization of the corpus required approximately 3 hours of continuous execution. This highlights a classic text mining trade-off: sacrificing processing speed in favor of maximizing the structural and semantic quality of the dataset.

Following the computationally expensive neural processing executed by Stanza, the resulting lemmatized dataset was serialized and saved to disk. This architectural decision guarantees that the heavy NLP pipeline which took approximately 3 hours to run does not need to be re-executed during the subsequent machine learning and analytical phases. The code block under analysis represents the critical transition between textual pre-processing and formal mathematical text representation.

3.3 Text Representation

Machine learning algorithms and mathematical similarity metrics cannot natively process raw string formats. To bridge this gap, the textual data must undergo vectorization, a foundational process in Natural Language Processing (NLP) that maps the linguistic corpus into structured mathematical matrices. To implement this, we utilized **scikit-learn**, (Pedregosa et al., 2011) the industry-standard machine learning library for Python, specifically leveraging its feature extraction modules. The overarching theoretical framework applied here is the **Vector Space Model (VSM)** (Salton et al., 1975), which represents each document (in our case, a track’s lyrics) as an algebraic vector in a multidimensional space, where each dimension corresponds to a distinct, unique lemma from the global vocabulary.

Before computing the final matrices, it is imperative to define strict statistical thresholds to filter out residual noise. Even after the aggressive grammatical pruning performed during the POS-tagging phase, a domain-specific corpus naturally contains term distributions that offer no predictive value. In Information Retrieval, vocabulary size grows rapidly, often leading to the Curse of Dimensionality, a phenomenon where excessive features cause models to overfit, drastically increasing computational memory overhead without yielding better analytical results. To proactively mitigate this, we constrained the vocabulary using document frequency boundaries:

- **Maximum Document Frequency (max_df=0.75):** This threshold establishes an upper bound, automatically discarding terms that appear in more than 75% of the documents. From an information-theoretic standpoint, terms with extremely high frequency carry low entropy (Information Content); they are predictable and lack discriminative power. In the context of musical lyrics, overly ubiquitous words (e.g., “amore”, “cuore”) effectively act as domain-specific stop-words. Because their distribution is so uniform across the corpus, they fail to highlight distinct thematic clusters and are therefore safely discarded.
- **Minimum Document Frequency (min_df=0.01):** Conversely, this parameter enforces a lower bound, eliminating words that appear in fewer than 1% of the tracks. According to Zipf’s Law (Zipf, 1949), a large portion of any linguistic corpus consists of the “long tail”, extremely rare terms, hapax legomena, idiosyncratic slang, or transcription typos. Removing these rare anomalies is crucial for reducing matrix sparsity and preventing the subsequent machine learning algorithms from modeling statistical noise.

With the boundaries of our vocabulary strictly defined, we generated two distinct mathematical representations of the corpus to enable downstream comparative analysis.

The Bag-of-Words (BoW) Model (CountVectorizer)

It is the most straightforward representation of text data. It scans the filtered corpus and tallies the absolute frequency of each retained lemma. The resulting output is a high-dimensional, sparse matrix where the rows correspond to individual documents (the tracks) and the columns represent the discrete frequency counts of the vocabulary terms. While BoW effectively captures local word prominence, it is inherently agnostic to the global context of the corpus.

Term Frequency-Inverse Document Frequency (TfidfVectorizer)

To address the limitations of absolute frequency counts, we implemented the TF-IDF representation. TF-IDF does not merely count occurrences; it weights them. It is designed to evaluate how relevant a specific term is to a particular document, relative to the entire corpus. The metric is computed as the product of two statistics:

- **Term Frequency (TF):** Measures the local importance of a term t within a specific document d .

- **Inverse Document Frequency (IDF)**: Measures the global rarity of the term across the entire document set D , logarithmically scaling down the weight of terms that occur in many documents.

$$\text{TF-IDF}(t, d, D) = \text{tf}(t, d) \times \log\left(\frac{N}{|\{d \in D : t \in d\}|}\right)$$

where t represents the specific term (or lemma) being evaluated, d is the individual document (in our case, a single track), and D denotes the entire corpus. The variable N corresponds to the total number of documents within the corpus, while $|\{d \in D : t \in d\}|$ represents the Document Frequency, defined as the absolute number of documents containing the term t .

A critical architectural decision was made during the implementation of these vectorizers. The `TfidfVectorizer` is initialized explicitly with the vocabulary learned by the primary BoW vectorizer: `TfidfVectorizer(vocabulary=bow_vectorizer.vocabulary_)`.

By forcing the TF-IDF matrix to inherit the exact same feature space generated by the `CountVectorizer`, we mathematically guarantee that both the BoW and TF-IDF matrices possess identical dimensional shapes and identically ordered columns. This strict structural consistency prevents misaligned feature indices and dimensional mismatches. Consequently, it guarantees seamless interoperability, allowing us to feed these distinct matrices into downstream clustering algorithms or classification models to objectively compare their performance and evaluate the impact of term weighting versus absolute frequency.

4 Topic modeling

In this section, we address the topic modeling task, which allowed us to compare machine learning methods with neural networks and extract the main topics present in the songs.

4.1 LDA, NMF and LSA

For topic modeling, with the aim of understanding the main themes contained in the Italian songs, we started with three of the most well known algorithms: LDA, NMF, and LSA.

- **LDA (Latent Dirichlet Allocation)** (Blei et al., 2003): A generative probabilistic model assuming that documents are random mixtures over latent topics, and topics are distributions over words. Because it fundamentally models the probability of discrete events (word occurrences), **LDA must be fed the absolute integer counts from a Bag of Words (BoW) matrix**, rather than the continuous weights of TF-IDF. This approach excels at capturing broad, frequent term co-occurrences.
- **NMF (Non-Negative Matrix Factorization)** (Lee & Seung, 1999): A linear algebraic model that factorizes high-dimensional vectors into a lower-dimensional space while strictly enforcing non-negativity. This purely additive nature, combining parts to form a whole without subtraction, makes the resulting topics highly coherent and interpretable. NMF performs optimally when fed the weighted, continuous values of our **TF-IDF matrix**, allowing it to identify highly specific and distinctive themes.
- **LSA (Latent Semantic Analysis / TruncatedSVD)** (Deerwester et al., 1990): A classic dimensionality reduction technique based on Singular Value Decomposition (SVD). While it does not enforce non-negativity (meaning terms can have negative associations with a topic, essentially defining a

topic by what it is *not*), LSA is highly efficient at capturing semantic variance and resolving issues like synonymy. Like NMF, it yields the best results when utilizing the **TF-IDF matrix**.

Finding the optimal number of topics (K) in Machine Learning is rarely an exact science; it requires empirical testing. To determine the most effective mathematical approach for our specific corpus, we implement a **Grid Search** evaluating three distinct dimensionality reduction algorithms LDA, NMF, and LSA across a range of K values (from 3 to 10).

Because unsupervised learning lacks a definitive “ground truth” to measure against, we cannot rely on standard accuracy metrics. Instead, we must evaluate the quality of the generated topics using two custom natural language metrics:

- **Topic Diversity:** A model might achieve high coherence by generating multiple identical topics, which provides no analytical value. Diversity counters this by calculating the proportion of unique words across all top words extracted by the model.

$$\text{Diversity} = \frac{|\text{Unique Words}|}{\text{Total Words extracted}}$$

A score closer to **1.0** indicates that the model successfully separated distinct concepts without overlapping vocabularies.

- **UMass Topic Coherence** (Mimno et al., 2011): This metric measures the semantic quality of a topic by checking how often its top words actually co-occur in the same documents. It operates under the assumption that words belonging to a coherent concept will frequently appear together in the original text. The algorithm computes the co-document frequency $D(w_i, w_j)$ of word pairs, applying smoothing to prevent mathematical errors. The formula is expressed as:

$$C_{\text{UMass}} = \sum_{i>j} \log \left(\frac{D(w_i, w_j) + 1}{D(w_j)} \right)$$

Because it calculates a logarithmic penalty for words that rarely co-occur, the resulting score is strictly negative. A score **closer to 0** indicates higher semantic coherence.

Topic modeling is computationally expensive. Running three algorithms across eight different K configurations sequentially would be slow. To solve this, we leverage the **joblib** library (Varoquaux et al., 2009). By wrapping our evaluation function in `Parallel(n_jobs=-1)`, we command Python to bypass the Global Interpreter Lock (GIL) and distribute the training tasks concurrently across 100% of the available CPU cores. Furthermore, we pre-calculate a binary boolean mask of our BoW matrix before launching the parallel workers; this drastically accelerates the matrix dot-products required for the UMass Coherence calculations.

Finally, to make sense of this multidimensional evaluation and objectively determine the best model using intrinsic metrics, we pass the results to **Plotly**. This generates an interactive, colorblind-friendly dashboard that allows us to visually balance the trade-off between Coherence and Diversity to make our final selection.

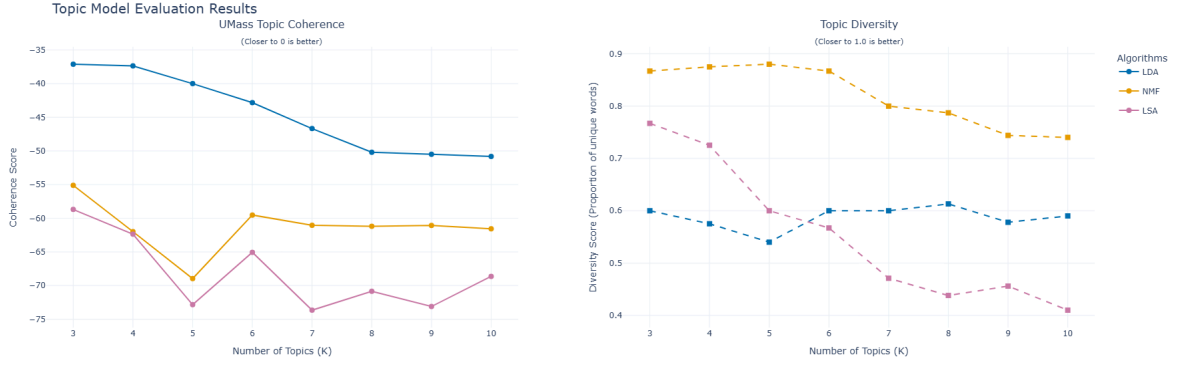


Figure 1: Confronting LDA, NMF and LSA

Based on the visual evaluation of our metrics, we selected **NMF** with $K = 6$ as our final model because it provides the optimal trade-off between semantic coherence and vocabulary diversity.

While LDA achieves the best (closest to zero) UMass Coherence scores, its Topic Diversity is critically low (hovering around 0.6), indicating that it generates highly overlapping and redundant topics. Conversely, NMF strongly dominates the diversity metric. At exactly $K = 6$, NMF maintains a near-peak diversity score (approximately 0.87) while simultaneously showing a strong recovery in coherence compared to the sharp dip observed at $K = 5$. This specific configuration ensures that our extracted themes are both logically consistent and distinctly separated.

4.1.1 Topic Extraction and Interpretation: Implementing NMF

Once the optimal number of topics has been identified through our grid search evaluation, we can proceed with the final extraction. For this phase, we isolate **Non-Negative Matrix Factorization (NMF)** as our primary algorithm.

NMF is a powerful linear algebraic method that approximates our high-dimensional TF-IDF matrix (V) by factoring it into two lower-dimensional matrices.

Because NMF enforces a strict non-negativity constraint ($W \geq 0, H \geq 0$), it does not allow for subtractive relationships. Topics are constructed purely by *adding* semantic parts together. This additive property mirrors how human language naturally constructs meaning, making NMF topics highly interpretable.

Here is a methodological breakdown of the code:

- **Hyperparameter Selection:** We define `NUM_TOPICS = 6` (our K dimensionality) as the empirical baseline for our specific domain, alongside `NO_TOP_WORDS = 10` to restrict our analytical focus to the most dominant features of each latent topic.
- **Deterministic Initialization (`init='nndsvd'`):** Instead of initializing the matrices with random noise, we use Nonnegative Double Singular Value Decomposition (NNDSDVD). This is a specialized initialization technique designed explicitly for sparse matrices (like our TF-IDF matrix). It guarantees deterministic results (the same output every run, regardless of the random seed) and drastically accelerates mathematical convergence.
- **The Extraction Logic (`model.components_`):** In scikit-learn, the `components_` attribute corresponds to the H matrix (the topic-term matrix). Its shape is exactly $K \times N$ (Number of Topics $\times$ Vocabulary).

Size). Each row represents a topic, and each column contains the specific weight of a word within that topic.

- **Sorting and Mapping (argsort):** because the H matrix only contains numerical weights, the `display_topics` function leverages NumPy's `argsort()` to rank the indices of these weights in ascending order. By reversing the array and slicing the top N indices, we can map these numbers back to the original string tokens stored in `feature_names`, revealing the human-readable vocabulary that defines each topic.

By simultaneously printing the exact numerical weights alongside the top words, we gain insight into the *distribution* of a topic. A topic where the top word has a weight of 2.5 while the second has 0.3 is heavily anchored to a single concept. Conversely, a topic with smoothly decaying weights (e.g., 1.2, 1.1, 0.9) indicates a broader, more nuanced semantic theme.

Topic	Words and weight per topic									
1	più 2.10	solo 1.03	mai 0.85	ancora 0.81	sempre 0.75	giorno 0.75	notte 0.74	mondo 0.72	qui 0.71	sentire 0.71
2	amore 3.33	amare 0.72	cuore 0.59	bello 0.34	morire 0.29	fiore 0.24	dolce 0.22	donna 0.19	cantare 0.19	grande 0.19
3	avere 2.06	soldo 0.92	stare 0.86	cazzo 0.73	chiamare 0.72	mettere 0.67	prendere 0.61	solo 0.61	amico 0.60	casa 0.58
4	sapere 3.21	dire 1.49	mai 0.85	amare 0.71	stare 0.42	pensare 0.40	solo 0.32	già 0.32	anche 0.30	cosa 0.30
5	volere 2.34	tenere 0.94	nun 0.90	cchiù 0.86	mo 0.73	aggio 0.62	pecché 0.58	ancora 0.46	pure 0.45	vita 0.42
6	andare 3.05	bene 0.93	via 0.73	così 0.53	dove 0.49	mare 0.37	stare 0.35	venire 0.28	lasciare 0.27	dire 0.26

Table 1: Result from NMF ($K = 6$).

4.2 Neural Topic Modeling: Building a VAE with PyTorch

While algorithms like LDA and NMF are excellent, Deep Learning allows us to capture much more complex relationships in text. In this section, we transition to **PyTorch** to build a neural model based on the **Variational Autoencoder (VAE)** (Srivastava & Sutton, 2017) architecture.

Here is a breakdown of the logical steps and key implementation choices behind our code:

- **Strict Reproducibility:** Neural networks on GPUs are inherently chaotic and stochastic. To ensure the model produces *exactly the same topics* every time we run the notebook, we locked down every single source of randomness (fixing the seeds (42) for NumPy, PyTorch, and CUDA). This is crucial for scientifically valid results.
- **Hyperparameter Alignment (Why 6 Topics?):** We explicitly set `NUM_TOPICS = 6`. We chose this specific number because it was identified as the optimal value for NMF during our earlier Grid Search evaluation. By keeping the dimensionality identical, we establish a level playing field, allowing us to directly and fairly compare the semantic quality of traditional linear algebra (NMF) versus Deep Learning (VAE).
- **The architecture (encoder & decoder):**

- **The encoder** takes our text and compresses it into a smaller “latent space.” Instead of turning a document into a simple vector of numbers, it transforms it into a *probability distribution*. This makes the model much more flexible and robust when handling ambiguous words.
- **The decoder** does the exact opposite: it takes these compressed concepts and tries to reconstruct the original words. The “weights” the decoder uses to rebuild the words act, functionally, as our topic matrix. By extracting these weights at the end of training, we can read our human-interpretable topics.
- **The Loss Function:** The model learns by trying to minimize an error metric called the ELBO. Essentially, it does two things simultaneously: it tries to reconstruct the original text as accurately as possible, but at the same time, it “restricts” itself (regularization) to prevent the network from simply memorizing the dataset.
- **Smart Training (Early Stopping):** Instead of guessing a fixed number of “epochs,” we implemented an *Early Stopping* mechanism. The model monitors itself: if it fails to improve for 10 consecutive cycles, it stops automatically and recovers its best saved state. This saves precious GPU time and prevents overfitting.

We loaded the **Bag-of-Words (BoW)** matrix into the neural network, whereas we used TF-IDF for NMF. This is not a mistake, but a deliberate architectural choice. Neural models like this are probabilistic generative models: they need to “count” actual word occurrences (discrete integers). Feeding them the fractional, continuous weights of a TF-IDF matrix would violate the mathematical rules they are built upon, severely degrading the quality of the extracted topics.

After 113 epoch these are the results from the neural network.

Topic	Words and Weights (Decoder Weights)									
1	cchiù 6.873	aggio 6.576	nun 6.290	pecché 6.287	cu 5.525	mo 5.522	quanno 5.328	tenere 5.171	votare 5.015	sordo 4.894
2	gang 5.813	soldo 5.282	troia 4.429	cazzo 4.366	negro 4.193	avere 3.893	bitch 3.783	rapper 3.666	spendere 3.617	chiamare 3.577
3	libero 4.162	terra 3.631	libertà 3.478	sangue 3.451	uomo 3.450	anima 3.417	vivo 3.394	mondo 3.369	fuoco 3.134	nero 3.070
4	ballare 6.105	notte 5.455	luna 5.187	mare 4.559	giù 4.446	estate 4.423	andare 4.027	piovere 4.024	sole 3.861	blu 3.796
5	amore 7.551	amare 6.807	donna 5.431	innamorare 5.290	bello 4.909	volere 4.874	cuore 4.623	dolce 4.563	più 4.518	mai 4.263
6	solo 4.607	mai 4.568	pensare 4.435	volta 4.430	sapere 4.393	perdere 4.359	dire 4.134	stare 4.022	forse 3.859	male 3.795

Table 2: Results from Neural Topic Modeling (VAE, $K = 6$).

We used the trained neural network to process the lemmatized tracks and categorize each one into one of the six distinct topics. While this step might initially seem redundant, it will prove highly useful for the subsequent text classification phase.

The output includes the following fields:

- **topic_word**: The representative keyword associated with the assigned topic (i.e., the primary keyword defining that specific theme).
- **topic_confidence_%**: The model's confidence score (expressed as a percentage) regarding the topic assignment.

This phase of the analysis reveals a notable limitation of the neural network: its struggle to fully grasp the semantic nuances and idiomatic flexibility of the Italian language. Because the model lacks human-like contextual comprehension, it occasionally misclassifies tracks that rely heavily on metaphors.

For instance, taking Calcutta's song "Tutti" is assigned to the **ballare** (dance/summer) topic, despite the lyrics actually exploring a nostalgic, unrequited love. Similarly, the track "Hubner" is also miscategorized under the same dance topic. In reality, the song relies on subtle storytelling, using the figure of soccer player Dario Hubner who famously rejected a highly lucrative contract to remain close to his wife as a metaphor for love.

4.3 Comparative Analysis: NMF vs. Neural VAE

While both models successfully capture broad thematic categories, the **Neural VAE demonstrates a significantly higher capacity to isolate specific musical subgenres, cultural tropes, and modern slang**, whereas NMF tends to cluster around common syntactical structures and high-frequency verbs.

Both algorithms successfully identified the three most dominant pillars of the Italian music corpus: Traditional Romance, Neapolitan dialect, and Urban/Rap music. However, they approach these themes differently.

The "Classic Love Song"

- **NMF Topic 2**: *amore, amare, cuore, bello, morire, fiore, dolce...*
- **VAE Topic 5**: *amore, amare, donna, innamorare, bello, volere, cuore...*

Both models perfectly capture the quintessential Italian pop/love vocabulary. The clusters are nearly identical, showing that romantic terminology is the most statistically dominant and cohesive semantic field in the dataset regardless of the underlying algorithm.

The Regional / Neapolitan Core

- **NMF Topic 5**: *volere, tenere, nun, cchiù, mo, aggio, pecché...*
- **VAE Topic 1**: *cchiù, aggio, nun, pecché, cu, mo, quanno, tenere...*

Because Neapolitan dialect acts almost as a distinct language within the dataset, both models successfully isolate it. However, the VAE cluster is slightly purer; NMF includes standard Italian verbs (*volere, tenere, ancora*) mixed with dialect, whereas the VAE extracts a strictly dialectal lexicon.

Rap, Trap, and Urban Slang

- **NMF Topic 3**: *avere, soldo, stare, cazzo, chiamare, mettere...*
- **VAE Topic 2**: *gang, soldo, troia, cazzo, negro, avere, bitch, rapper...*

This is where the Neural VAE shows a massive leap in semantic understanding. NMF identifies street slang (*soldo, cazzo*) but mixes it with generic high-frequency verbs (*avere, stare, mettere*). The VAE

aggressively isolates the modern Trap/Rap subgenre, cleanly grouping explicit terminology alongside English loanwords (*gang, bitch, rapper*) that are highly specific to the contemporary urban scene.

Where Neural VAE Outperforms NMF (Semantic Depth)

The remaining topics highlight the limitations of NMF's linear algebraic approach compared to the VAE's non-linear, contextual embeddings. NMF groups words by syntactic co-occurrence (often clustering generic verbs), while the VAE forms distinct, culturally recognizable musical tropes.

The “Tormentone Estivo” (Summer Hit / Dance)

- **VAE Topic 4:** *ballare, notte, luna, mare, giù, estate, andare, piovere, sole, blu*
- **NMF Equivalent:** Scattered across Topic 1 and 6.

The VAE beautifully identifies the classic Italian summer dance track. It clusters highly evocative, atmospheric nouns (*estate, mare, luna, sole*) and actions (*ballare*). NMF fails to consolidate this theme, splitting words like *mare* and *notte* into broad, generic spatial/temporal topics.

Conscious / Epic / Protest Music

- **VAE Topic 3:** *libero, terra, libertà, sangue, uomo, anima, vivo, mondo, fuoco, nero*
- **NMF Equivalent:** Completely missed.

The VAE extracts a highly cohesive cluster related to struggle, freedom, and epic/political narratives (e.g., conscious rap, folk, or classic singer-songwriter tracks). NMF's linear factorization could not isolate this relatively niche but semantically dense vocabulary.

Introspection and Melancholy

- **VAE Topic 6:** *solo, mai, pensare, volta, sapere, perdere, dire, stare, forse, male*
- **NMF Topics 1, 4, 6:** *più, solo, mai, ancora... / sapere, dire, mai, amare...*

While NMF creates several overlapping, verb-heavy clusters representing general communication and existence (Topics 1, 4, and 6), the VAE consolidates the emotional weight into a single introspective topic focusing on mental state, doubt, and loss (*pensare, perdere, forse, male*).

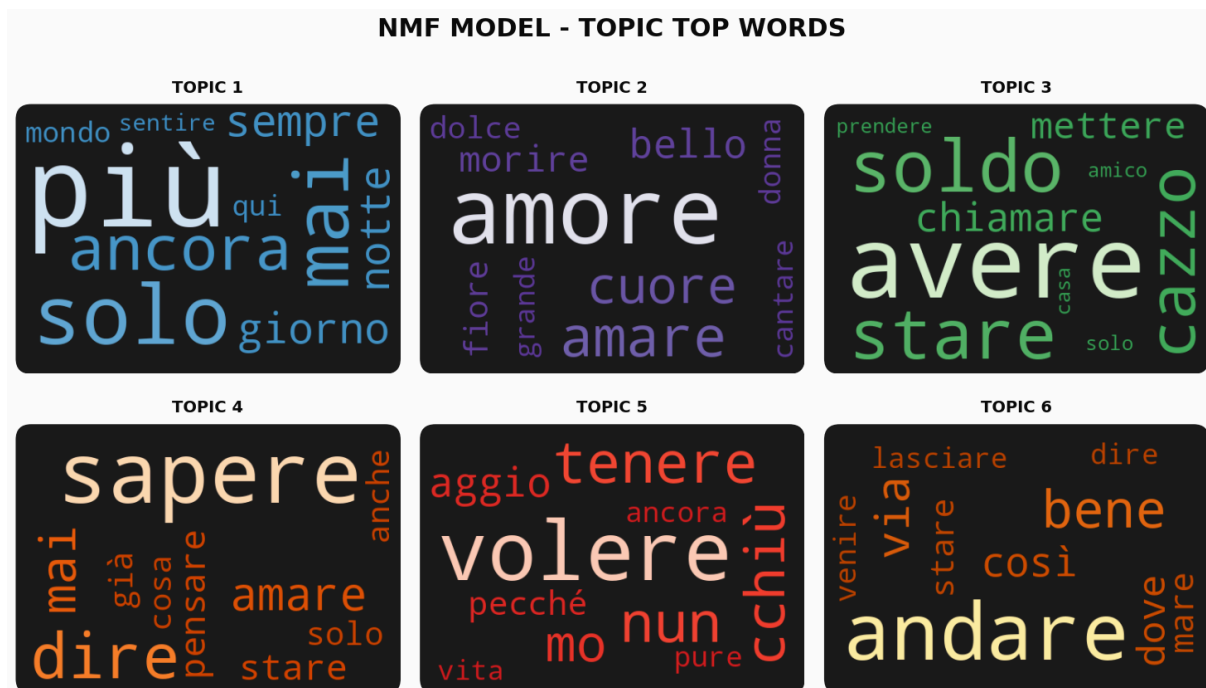


Figure 2: Topic top words NMF



Figure 3: Topic top words VAE

5 Text Classification

In this section, we address the text classification task, which allowed us to compare machine learning methods with neural networks and categorize the songs into predefined classes.

Text classification is a supervised machine learning task aimed at automatically assigning a natural language document to one or more predefined categories based on its content. In the context of this

project, our primary objective is to train a predictive algorithm capable of deducing the broader musical macro-genres of an artist relying entirely on their tracks.

Traditional text classification often assumes mutual exclusivity, where a document is assigned to a single, distinct category. However, music is intrinsically hybrid and fluid. A single track or artist can frequently bridge distinct musical traditions, simultaneously belonging to multiple macro-genres (e.g., an artist classified as both ‘Indie’ and ‘Pop’). Therefore, formulating this as a standard multi-class problem would force an artificial mutual exclusivity, leading to unacceptable data loss. To accurately reflect this overlapping reality, the predictive architecture must be structured as a **multi-label classification** task, where the algorithm evaluates and assigns multiple genre labels to a single textual instance.

5.1 Genre Pre-Processing

In this phase, we import the track and artist metadata, alongside the lemmatized linguistic corpus generated in the previous steps. However, a preliminary inspection of the dataset revealed a significant structural challenge: the Last.fm API associates each artist with their top 5 “most voted” genres, resulting in a staggering total of 1,110 distinct genre tags across our corpus. Raw metadata extracted from collaborative digital platforms, like Last.fm, is notoriously noisy. It is structured as a “folksonomy”: An unstructured, user-generated web of highly granular, idiosyncratic micro-genres (e.g., eurodance, synthwave, downtempo). From a machine learning perspective, feeding over a thousand fragmented, low-frequency tags into an analytical model leads to extreme dimensionality and data sparsity. To resolve this and create a mathematically manageable target space, we implemented a rule-based taxonomy system designed to aggregate these hundreds of micro-genres into eight standardized, macro-categorical buckets. To execute this dimensionality reduction, we engineered a custom classification script centered around a heuristic dictionary and efficient Python data structures:

- **The Heuristic Dictionary (PARENT_GENRE_RULES):** We defined a structured dictionary where the keys represent our eight target macro-genres (e.g., ‘Rock’, ‘Electronic’, ‘Cantautore’) and the values are lists of carefully selected substring keywords. By employing substring matching, the algorithm can capture dozens of highly specific variations with a single rule. For instance, the keyword ‘metal’ will automatically and correctly capture tags like ‘heavy metal’, ‘nu metal’, and ‘black metal’, routing them to the ‘Rock’ parent category.
- **Handling Unmapped Outliers:** It is statistically inevitable that certain irrelevant tags will fail to match any of our predefined rules. Rather than forcing these outliers into arbitrary buckets and contaminating our dataset, the algorithm isolates them into an `unmapped_genres` array (the leftovers). These unmapped tags are then effectively discarded. This strict filtering mechanism guarantees that our final classified dataset remains semantically pure and focused entirely on the predefined analytical scope.

The most critical architectural choice in the classification script is the intentional omission of the `break` statement after finding a keyword match. Standard algorithms usually assume mutual exclusivity, but music is inherently hybrid; for example, a micro-genre like “indie pop” conceptually belongs to both the ‘Indie’ and ‘Pop’ macro-categories. By allowing the iteration to continuously evaluate all rules, a single micro-genre can be dynamically assigned to multiple parent categories, creating a polyhierarchical taxonomy. This approach accurately reflects the overlapping reality of musical genres and prevents arbitrary data loss by ensuring hybrid tracks are correctly represented in all relevant analytical clusters.

However, because this multiple-mapping strategy can inadvertently process the same sub-genre string multiple times, a final cleaning step is necessary. To resolve this, the output generation utilizes the specific Python idiom `list(dict.fromkeys(child_genres))`. This method implicitly deduplicates the dataset while strictly preserving the original deterministic sorting, which is essential for ensuring reproducibility in the final dataframes.

Following this methodology, each artist was successfully mapped to one or more broader macro-genres. However, entities lacking any valid genre metadata were systematically pruned from the dataset to preserve data integrity and prevent `null` values from compromising downstream analytical tasks.

5.2 Machine Learning

To tackle the complexities of this multi-label text mining problem, we benchmarked three distinct predictive architectures. Each algorithm was selected to evaluate a different mathematical approach to high-dimensional data and label correlation:

- **Support Vector Machines via One-vs-Rest (`OneVsRestClassifier` + `LinearSVC`):** Support Vector Machines (SVMs) are highly effective for text classification tasks. Because text vectorization (like TF-IDF) generates extremely high-dimensional and sparse feature matrices, SVMs excel by finding the optimal hyperplane that separates classes while resisting the Curse of Dimensionality. Since standard SVMs are inherently binary classifiers, we utilized a One-vs-Rest (OvR) strategy. This wrapper trains a separate, independent binary classifier for every single macro-genre, evaluating whether a track belongs to a specific genre or not, and aggregates the results into a multi-label prediction.
- **Linear Classifier Chains (`ClassifierChain` + `LinearSVC`):** While the OvR approach is highly efficient, it evaluates each genre in complete isolation, ignoring potential statistical correlations between labels (e.g., the high probability that a ‘Hip-Hop’ track might also trigger ‘Pop’ features). The Classifier Chain architecture addresses this limitation. It links several binary SVM classifiers in a sequence; the prediction made by the first classifier in the chain is appended as a new feature for the next classifier. This allows the model to leverage label dependencies and contextual relationships, potentially improving predictive accuracy on highly correlated genres.
- **Tree-Based Classifier Chains (`ClassifierChain` + `XGBClassifier`):** Moving away from linear hyperplanes, we tested Extreme Gradient Boosting (XGBoost), a tree-based ensemble algorithm. Gradient boosting builds a sequence of decision trees, where each subsequent tree is trained specifically to correct the residual errors of the previous ones. To maximize its effectiveness for our multi-label task, we encapsulated the XGBoost classifier within a `ClassifierChain`. This hybrid architecture leverages both XGBoost’s exceptional ability to capture complex, non-linear relationships within the lexical feature space, and the chain’s capacity to model statistical correlations between overlapping genres.

Algorithm default settings are rarely optimal for highly specialized, domain-specific text corpora. To ensure that our models reached their absolute mathematical optimum, we implemented an exhaustive hyperparameter tuning phase using `GridSearchCV`.

Building upon the vocabulary constraints established in [Section 3.3](#), the text representation for the machine learning models requires carefully optimizing the feature space. Because the classification process relies on representing texts as vectors of features, it is crucial to select parameters that yield the highest discriminative power while managing computational complexity. To achieve this, instead

of relying on a static configuration, the tuning process explores different dimensions of the feature space in a continuous prose format.

The grid also fine-tunes document frequency filters to isolate words within the most useful intermediate frequency range. The `max_df` parameter is restricted to 0.70, serving as an upper cut-off to eliminate highly frequent, ubiquitous terms that lack the descriptive power needed to differentiate between classes. Conversely, a `min_df` of 10 establishes a strict lower bound based on absolute document occurrence rather than a percentage, effectively removing extremely rare terms and anomalies to reduce matrix sparsity. Finally, the grid evaluates the core weighting scheme itself by toggling the `use_idf` parameter. This step empirically tests whether weighting terms inversely to their broader document frequency genuinely improves the model's predictive accuracy, or if relying solely on raw term frequencies is sufficient for the given corpus.

5.2.1 Support Vector Machines via One-vs-Rest

To establish a strong, highly interpretable baseline for our multi-label classification task, we implemented a Support Vector Machine architecture using Scikit-Learn. The methodology is divided into three distinct phases:

- **Data Preparation and Target Binarization:** Prior to machine learning processing, the dataset undergoes necessary restructuring. First, lemmatized lyrics are concatenated at the artist level to create a single, unified document string for each artist. Next, to handle the fact that artists often span multiple musical categories, a `MultiLabelBinarizer` is used to encode the target variables into a dense binary matrix, clearly indicating the presence or absence of each macro-genre. Finally, a standard 80/20 train-test split is applied to the dataset to guarantee an unbiased evaluation of the final model on unseen data.
- **Pipeline:** To ensure that all textual transformations are uniformly applied during both training and testing, the processing architecture is encapsulated within a standardized Scikit-Learn Pipeline. The pipeline consists in two steps:
 - `TfidfVectorizer`: This component translates the concatenated raw text into a mathematically workable format, representing each artist as a sparse numerical vector based on Term Frequency-Inverse Document Frequency (TF-IDF).
 - `OneVsRestClassifier` + `LinearSVC`: The predictive engine is a Linear Support Vector Classifier. Crucially, we enforce `class_weight='balanced'` to dynamically adjust the algorithm's internal loss function, heavily penalizing misclassifications in underrepresented musical genres (minority classes) while softening the penalty for ubiquitous genres. Furthermore, `max_iter` is increased to 12,000 to ensure mathematical convergence during the optimization of the hyperplanes. Because `LinearSVC` natively supports only binary or standard multi-class formats, it is wrapped in a `OneVsRestClassifier`, which independently fits a binary model for every single macro-genre.
- **Hyperparameter Optimization Space:** To maximize the model's predictive accuracy, a comprehensive Grid Search was executed to optimize both text vectorization and linear classification boundaries. During the vectorization phase, the setup balanced computational efficiency and lexical richness by fine-tuning dimensionality (`max_features`), contextual framing (`ngram_range`), and weighting strategies (`use_idf`). On the classification side, the optimization focused on the regularization penalty (`C`) to control the bias-variance tradeoff, alongside testing different loss functions (`loss`) to calculate boundary errors optimally. The entire process was run using 3-fold cross-validation and evaluated against the Macro-Averaged F1-Score (`scoring='f1_macro'`), a metric chosen specifically to ensure the model treats rare musical genres with the same importance as ubiquitous

ones. Finally, to rapidly process the thousands of model fits required, the workload was aggressively parallelized (`n_jobs=-1`) to utilize all available CPU cores.

	precision	recall	f1-score	support
Cantautore	0.35	0.75	0.48	36
Classical	0.30	0.90	0.45	30
Electronic	0.36	0.52	0.42	56
Hip-Hop	0.66	0.83	0.74	64
Indie	0.49	0.61	0.54	36
Jazz	0.24	0.67	0.36	30
Pop	0.58	0.67	0.62	76
Rock	0.46	0.42	0.44	73
micro avg	0.43	0.65	0.51	401
macro avg	0.43	0.67	0.51	401
weighted avg	0.46	0.65	0.53	401
samples avg	0.46	0.68	0.52	401

Table 3: Classification report, OneVsRestClassifier LinearSVC

5.2.2 Linear Classifier Chains

While the One-vs-Rest (OvR) strategy provides a highly efficient and interpretable baseline, it suffers from a fundamental mathematical limitation: it treats every musical genre as entirely independent. In reality, musical genres are highly correlated. For example, a track exhibiting ‘Hip-Hop’ lexical features has a higher statistical probability of also containing ‘R&B’ elements, while simultaneously having a near-zero probability of being ‘Classical’. To capture and leverage these underlying semantic relationships, we implemented a Classifier Chain architecture.

The core of this pipeline remains the `TfidfVectorizer` paired with a `LinearSVC`. However, instead of evaluating each macro-genre in absolute isolation, the `ClassifierChain` links the binary SVM models in a strict mathematical sequence. The mechanism operates by cascading information: the first classifier in the chain trains on the standard TF-IDF matrix to predict the first genre. The second classifier then trains on both the TF-IDF matrix and the prediction output of the first classifier. This process continues down the chain. By appending previous predictions as new, synthetic features for subsequent models, the architecture actively learns contextual dependencies and label correlations, allowing it to make more informed decisions on overlapping musical genres.

Because the feature space expands with every link in the chain, and because the errors of early classifiers can propagate to later ones, this architecture demands higher mathematical stability from its base estimators. To accommodate this, we significantly increased the computational threshold of the Support Vector Machine (`max_iter=50000`). This ensures that the linear hyperplanes successfully converge and find the optimal decision boundaries, even when dealing with the augmented, chained feature spaces. As with the OvR approach, we maintained the `class_weight='balanced'` parameter to continuously penalize majority-class bias.

We subjected this architecture to the same exhaustive hyperparameter tuning using a 3-fold `GridSearchCV`, continuing to optimize for the Macro-Averaged F1-Score (`scoring='f1_macro'`) to guarantee fair evaluation across minority classes. However, hardware utilization becomes a critical

factor in this phase. Because the Classifier Chain inherently forces a strict, sequential, un-parallelizable training process for the genres within a single model fit, the overall training time increases substantially compared to OvR. Therefore, injecting `n_jobs=-1` at the Grid Search level is essential. While the chain itself must run sequentially, the Grid Search aggressively parallelizes the evaluation of different hyperparameter permutations (e.g., testing different C penalties or `ngram_ranges`) across 100% of the available CPU cores, keeping the optimization phase computationally viable.

	precision	recall	f1-score	support
Cantautore	0.39	0.42	0.41	36
Classical	0.29	0.77	0.42	30
Electronic	0.31	0.07	0.12	56
Hip-Hop	0.65	0.81	0.72	64
Indie	0.73	0.53	0.61	36
Jazz	0.40	0.13	0.20	30
Pop	0.61	0.63	0.62	76
Rock	0.44	0.32	0.37	73
micro avg	0.50	0.47	0.48	401
macro avg	0.48	0.46	0.43	401
weighted avg	0.49	0.47	0.45	401
samples avg	0.48	0.50	0.47	401

Table 4: Classification report, Classifier Chain LinearSVC

5.2.3 Tree-Based Classifier Chains (XGBoost)

While Support Vector Machines provide highly efficient and interpretable linear boundaries in high-dimensional sparse spaces, they inherently struggle to capture highly complex, non-linear semantic interactions between words. To explore non-linear decision boundaries while maintaining our contextual approach to musical genres, we introduced an advanced ensemble learning architecture: Extreme Gradient Boosting (XGBoost) (Chen & Guestrin, 2016) encapsulated within a Classifier Chain.

- **The Core Estimator (Gradient Boosting):** Unlike a single decision tree, XGBoost builds an ensemble of weak learners (shallow trees) sequentially. Each new tree is specifically trained to correct the residual errors made by the combination of all previous trees. For large-scale text classification, we utilize the `tree_method='hist'` parameter. This enables a histogram-based algorithm that bins continuous features, dramatically reducing computational time and memory consumption without sacrificing predictive accuracy.
- **Algorithmic Synergy with Text Data:** Text data is inherently sparse. XGBoost natively handles sparsity by learning optimal default directions for missing or zero-values at every tree split, making it highly compatible with our TF-IDF vectorization strategy.
- **Hyperparameter Tuning Strategy:** The tuning geometry for a Boosting model is fundamentally different from an SVM. Our GridSearchCV focuses on the delicate balance between learning capacity and regularization to prevent overfitting:
 - **Ensemble Size & Step Size (`n_estimators` & `learning_rate`):** These two parameters are strictly correlated. The `n_estimators` dictates the total number of sequential trees built. A higher number allows the model to learn more complex patterns but risks overfitting the training

data. The `learning_rate` shrinks the feature weights after every boosting step. We test various combinations because a robust model typically requires higher `n_estimators` paired with a lower `learning_rate`.

- **Tree Complexity (`max_depth`):** This defines the maximum number of nodes from the root to the farthest leaf. In text classification, shallow trees (e.g., depth 3) might fail to capture multi-word context, while overly deep trees (e.g., depth 9+) might memorize specific sentence structures.
- **Resource Management (`n_jobs` threading):** A critical architectural note for this pipeline is the management of parallel computing. Because XGBoost is highly optimized and manages parallel processing internally at the tree-construction level (`n_jobs=-1` inside the classifier), we explicitly restrict the Grid Search to a single thread (`n_jobs=1`). This prevents nested multiprocessing, which would otherwise lead to severe thread contention and memory bottlenecks.

Finally, as with previous iterations, the entire tuning process was strictly evaluated using the Macro-Averaged F1-Score (`scoring='f1_macro'`). This guarantees that the boosted trees are aggressively penalized if they ignore statistically rare minority classes (e.g., 'Classical' or 'Jazz') in favor of dominant, ubiquitous genres.

	precision	recall	f1-score	support
Cantautore	0.46	0.17	0.24	36
Classical	0.64	0.23	0.34	30
Electronic	1.00	0.04	0.07	56
Hip-Hop	0.78	0.72	0.75	64
Indie	1.00	0.11	0.20	36
Jazz	0.50	0.03	0.06	30
Pop	0.61	0.58	0.59	76
Rock	0.31	0.42	0.36	73
micro avg	0.54	0.35	0.42	401
macro avg	0.66	0.29	0.33	401
weighted avg	0.65	0.35	0.38	401
samples avg	0.50	0.39	0.42	401

Table 5: Classification report, XGBoost Chain

5.3 Transformer BERT

While traditional feature engineering techniques, such as Bag-of-Words (BoW) and Term Frequency-Inverse Document Frequency (TF-IDF), are computationally efficient and establish a highly interpretable baseline, they suffer from critical theoretical limitations, most notably the severe “loss of context”.

Furthermore, traditional lexical matching models are inherently incapable of resolving the semantic complexities of natural language, specifically synonymy (different words sharing the same underlying concept) and polysemy (a single word possessing multiple meanings depending on its usage). To overcome these representational bottlenecks and capture the deep semantic relationships within our lyrical corpus, we transitioned from sparse lexical matching to continuous semantic modeling, specifically employing a Deep Learning architecture based on Transformers.

To process the complex semantics of Italian lyrics, we utilized UmBERTo (Devlin et al., 2019; Liu et al., 2019; Musixmatch Research, 2020) (model Musixmatch/umberto-commoncrawl-cased-v1), an advanced language model developed by Musixmatch.

UmBERTo is built upon the RoBERTa architecture, which is itself a highly optimized evolution of Google’s foundational BERT (Bidirectional Encoder Representations from Transformers). This model was pre-trained on a massive dataset of Italian web pages extracted from the Common Crawl corpus. Because it was trained extensively on uncurated internet text, UmBERTo possesses a profound, native understanding of modern Italian syntax, grammatical rules, colloquialisms, and slang—elements that are ubiquitous in musical lyrics. Unlike traditional algorithms, UmBERTo utilizes self-attention mechanisms to read texts contextually, dynamically computing the meaning of a word based on its mathematical relationship with all surrounding words in the sentence.

Training Large Language Models (LLMs) on highly specific, imbalanced data requires strict architectural controls to prevent the network from simply memorizing the training set (overfitting). To maximize generalization and predictive accuracy, we implemented several advanced deep learning techniques throughout our pipeline.

Dataset Flattening and Stochastic Reproducibility

In our preliminary machine learning pipelines, we aggregated all lyrics at the artist level. However, feeding a Transformer a massive, concatenated string of 20 diverse songs causes severe “semantic blurring.” To resolve this, we flattened the dataset’s hierarchical structure so that 1 row = 1 independent track. This approach prevents the catastrophic loss of granular data, drastically multiplies the volume of available training samples, and provides the neural network with a much richer, track-specific vocabulary to associate with the inherited artist-level macro-genres.

Deep learning algorithms rely heavily on random weight initializations and stochastic batching. To guarantee that our pipeline is 100% deterministic and scientifically reproducible, we implemented strict seed locking. By explicitly fixing the random seeds across Python, NumPy, and PyTorch (for both CPU and CUDA environments), we ensured that our experimental results are consistent across multiple execution runs.

Strategic Tokenization: The Head-and-Tail Heuristic

Due to the quadratic memory complexity of the self-attention mechanism, Transformer models impose a strict hardware limit of 512 tokens per input sequence. A standard pop or rap song frequently exceeds this limit. Instead of utilizing default truncation algorithms which blindly slice off the end of a song, we engineered a semantic “Head+Tail” truncation strategy. A typical song structure consists of an introduction, verses, and a highly repetitive chorus in the middle. Our custom script explicitly extracts the first 150 words and the last 150 words of a track, intelligently bypassing the redundant center. This ensures that the neural network receives the most narrative and contextually rich segments of the text (the opening themes and the concluding remarks) without triggering memory overflow errors.

Anti-Overfitting Defenses: Layer Freezing and Early Stopping

UmBERTo already possesses a deep structural understanding of the Italian language. To prevent “Catastrophic Forgetting”, a phenomenon where a neural network overwrites and destroys its pre-trained linguistic knowledge while forcefully adapting to a new dataset, we implemented a Layer Freezing strategy. We explicitly froze the initial embedding representations and the first 6 encoder layers of the network. By locking these foundational layers, the network is only permitted to calculate gradients and update the weights of its highest, most abstract layers. This ensures a stable, focused

training phase specifically tailored to musical genre deduction.

To further suppress overfitting, we increased the `weight_decay` parameter to 0.05, applying strict L_2 regularization to penalize overly complex internal neural pathways. Furthermore, we integrated a Hugging Face `EarlyStoppingCallback` (Wolf et al., 2020). This mechanism continuously monitors the Validation Loss metric at the end of each epoch. If the network begins to memorize the training data and fails to improve for 2 consecutive epochs (`patience=2`), the training cycle is automatically aborted, and the algorithm restores the most generalized mathematical weights.

Algorithmic Compensation for Class Imbalance

Music taxonomies inherently follow a power-law distribution: mainstream genres like Pop or Hip-Hop drastically outnumber niche traditions like Jazz or Classical. If processed normally, the neural network would fall victim to the accuracy paradox, learning to exclusively guess the majority classes to minimize overall loss. To mathematically neutralize this bias, we subclassed the Hugging Face Trainer to inject a custom loss architecture based on Binary Cross Entropy with Logits (`BCEWithLogitsLoss`). Before training, we calculate the exact positive-to-negative ratio for every single macro-genre. These ratios are passed into the loss function as explicit Positive Weights (`pos_weights`). This mathematically distorts the gradient landscape: if the model misclassifies a statistically rare minority genre, it receives a massively magnified error penalty, forcefully guiding the network to pay equal attention to the entire musical spectrum.

Post-Training Calibration: Dynamic Threshold Optimization

By default, neural networks apply a rigid probability threshold of 50% when converting continuous sigmoid outputs (logits) into binary predictions (0 or 1). To extract the absolute maximum performance from our fine-tuned weights without retraining the model, we implemented a post-training Threshold Optimization loop. The script extracts the raw probability scores generated on the test set and performs a systematic grid search across a continuous range of thresholds (from 0.40 to 0.95). By empirically evaluating these cutoffs, the algorithm dynamically identifies the exact mathematical threshold that maximizes the Macro-Averaged F1-Score, ensuring our final `classification_report` reflects a highly calibrated sensitivity to both majority and minority musical genres.

	precision	recall	f1-score	support
Cantautore	0.38	0.71	0.50	625
Classical	0.46	0.50	0.48	141
Electronic	0.22	0.43	0.29	343
Hip-Hop	0.74	0.79	0.77	1058
Indie	0.35	0.64	0.45	513
Jazz	0.16	0.38	0.23	235
Pop	0.61	0.74	0.67	1165
Rock	0.33	0.65	0.44	768
micro avg	0.44	0.68	0.53	4848
macro avg	0.41	0.61	0.48	4848
weighted avg	0.48	0.68	0.56	4848
samples avg	0.49	0.71	0.55	4848

Table 6: Classification report BERT - UmBERTo optimized threshold 0.50

5.4 Comparative Analysis: ML vs. BERT

Before comparing the models, it is essential to understand the metrics used to evaluate their effectiveness.

Evaluation Metrics Definitions

In the context of multi-class classification, “Macro” metrics calculate the score for each class (genre) separately and then compute their unweighted mean. This means every class is treated equally, regardless of its frequency in the dataset.

- **Precision:** Answers the question: “*Out of all the songs the model predicted as belonging to a specific genre, how many actually belonged to it?*”. Mathematically, it is the ratio of True Positives to the sum of True Positives and False Positives. A high Precision indicates that the model produces very few “false alarms” (few false positives).
- **Recall (or Sensitivity):** Answers the question: “*Out of all the songs that actually belonged to a specific genre, how many did the model correctly identify?*”. It is the ratio of True Positives to the sum of True Positives and False Negatives. A high Recall indicates that the model successfully captures most of the correct instances, missing very few (few false negatives).
- **F1-Score:** Since there is often a trade-off between Precision and Recall (improving one usually degrades the other), the F1-Score provides a single unified metric. It is calculated as the *harmonic mean* of Precision and Recall. It is particularly useful when you need to strike a balance between the two metrics or when dealing with imbalanced datasets.

Global Commentary on the Four Models

Looking at the **Global Performance Comparison (Macro Averages)** at the bottom of the dashboard, distinct behavioral profiles emerge for each approach:

XGBoost Chain

This model achieves the highest **Macro Precision** (0.66) but the lowest **Macro Recall** (under 0.30). This means XGBoost Chain is extremely cautious: it only makes predictions when it is highly confident. Consequently, when it assigns a genre, it is usually right, but it completely misses the majority of the texts, generating a high number of false negatives. This imbalance leads to the lowest overall F1-Score (0.33).

BERT

The BERT neural network presents a different profile. It achieves a good **Macro Recall** (0.61, the second highest), but its **Macro Precision** is the lowest among the models (around 0.41). BERT manages to “catch” a large portion of the texts for each genre, but it produces more false positives compared to the other models. This suggests that for this specific lyrical text task, the BERT model might need more fine-tuning to refine its discriminative capabilities.

SVM Chain

The chained Support Vector Machine sits in the middle, offering a decent balance (Precision is 0.48 and Recall is 0.46). However, while it has better precision than BERT and SVM OvR, it is outperformed by both in terms of overall Macro F1-Score.

The Best Global Model: SVM OvR

Based on the global macro averages, the winner of this experiment is the **SVM OvR (Support Vector Machine via One-vs-Rest)** model. Looking at the global data, SVM OvR achieves the **Best Macro F1-Score** (reaching 0.50). The reason for its success on a macro level lies in its ability to find cases without sacrificing too much precision:

1. It achieves the highest **Macro Recall** (0.67), successfully finding the vast majority of texts belonging to the target classes.
2. At the same time, it maintains a slightly better Precision compared to BERT (0.43 vs 0.41).

A Nuanced Look at Per-Genre Performance

While SVM OvR wins on the global *macro* average, analyzing the data **per Musical Genre (Top Chart)** reveals a more complex reality:

- **BERT's Dominance in Standard Genres:** BERT actually achieves the highest F1-Score in the majority of individual categories: *Cantautore*, *Classical*, *Pop*, *Rock*, and *Hip-Hop*.
- **SVM OvR's Consistency on Difficult Genres:** The reason SVM OvR wins the overall Macro F1-Score is due to its superior robustness on the hardest genres to classify. SVM OvR stands out with significantly higher performances in *Electronic*, *Indie*, and *Jazz*, whereas BERT and the other models struggle severely (dropping below 0.30 or even 0.10 F1-scores).

(Note: Almost all models perform exceptionally well on Hip-Hop, likely due to highly distinct rhythmic text patterns and vocabulary, and struggle on Electronic and Jazz. Ultimately, BERT excels on the more populated/standard genres, but SVM OvR remains the most reliable and versatile option across the entire tested musical spectrum, preventing severe performance drops on the difficult classes.)

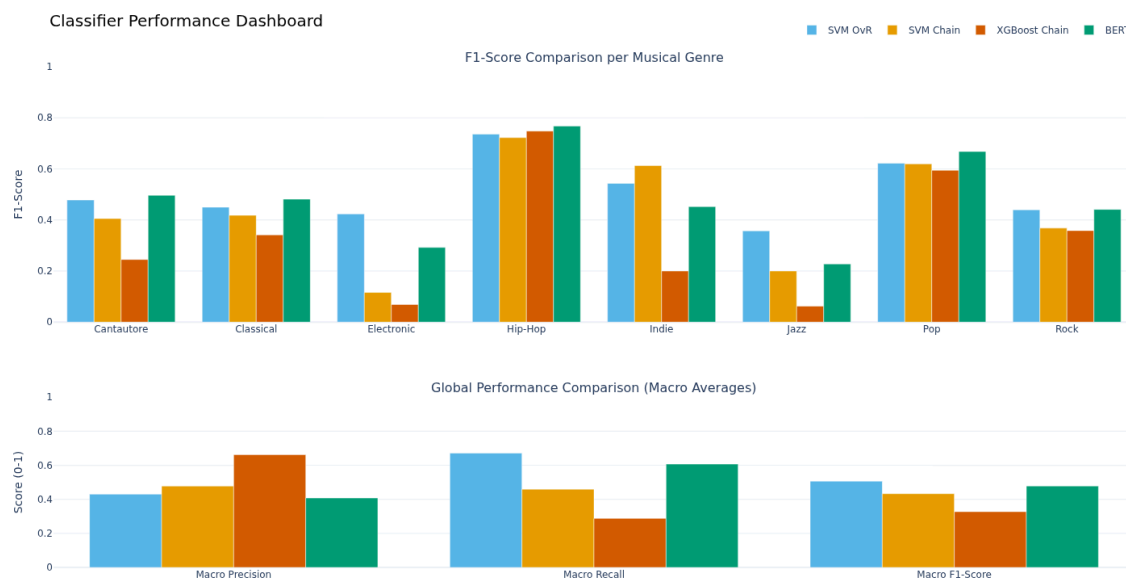


Figure 4: Comparison Classifier

5.5 Sentiment Analysis

Sentiment analysis is a Natural Language Processing (NLP) task aimed at automatically detecting the emotional state or polarity expressed within a span of text. By isolating songs classified under the dominant topic of “Love” (*Amore*), this analysis investigates the underlying emotional valence of these specific lyrics to determine whether Italian artists predominantly associate romance with joy or sorrow.

To mitigate the natural language ambiguity, the analysis employs FEEL-IT ([Bianchi et al., 2021](#)) (model MilaNLProc/feel-it-italian-sentiment), an advanced Transformer model specialized for the Italian language. Built upon the UmBERTo architecture, FEEL-IT was explicitly fine-tuned by researchers at the University of Milan Bocconi (MilaNLProc) to recognize emotion and sentiment in expressive and informal Italian texts.

To directly address the research question, the dataset is strictly filtered to include only tracks where the `topic_word` is assigned to `TARGET_TOPIC = "amore"`. When passing textual data to Transformer models, one must account for strict architectural limitations, such as the 512-token memory limit, the script utilizes the `lemmatized_lyrics` column that during the earlier text processing phases, lemmatization was applied to reduce words to their base morphological dictionary forms. Coupled with the removal of non-informative stop words (such as common articles and prepositions), this pre-processing naturally compresses the string length. Consequently, the entire informative payload of the lyrics can be passed to the model safely using vectorized batch processing, which maximizes computational efficiency.

The pipeline executes the inference and returns a structured list of dictionaries containing the predicted labels and their corresponding confidence scores.

The sentiment distribution reveals a striking insight into the thematic portrayal of love in Italian music:

- **A Melancholic Majority:** A significant 64.5% of the tracks categorized under the “Love” topic exhibit a Negative sentiment. This indicates a predominant lyrical focus on the painful aspects of romance, such as heartbreak, longing, betrayal, and unrequited love.
- **The Joyful Minority:** Only 35.5% of the songs are classified with a Positive sentiment, demonstrating that purely celebratory, happy, or fulfilled romantic narratives represent the minority within this corpus.

The data clearly illustrates a strong artistic tendency to utilize music as an emotional outlet for the complexities and sufferings associated with love. Within this dataset, artists are nearly twice as likely to explore the complications of romance rather than its triumphs. However, it must be acknowledged that automated text classification is subject to a degree of subjectivity and error. The rich semantic nuances, metaphors, and pragmatic context inherent in poetry and lyrics mean that while the overarching trend is highly robust, the exact classification of every individual track may vary slightly from human interpretation.

Overall Sentiment on Topic: 'AMORE'

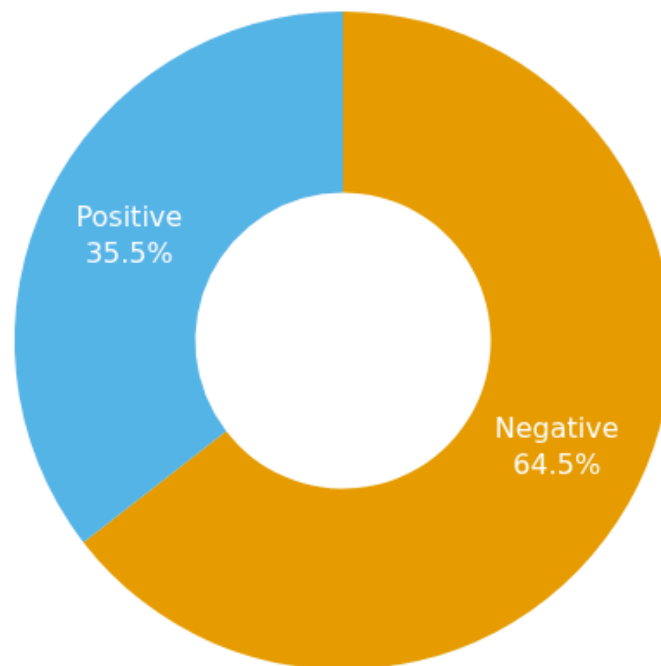


Figure 5: Sentiment analysis result

6 Conclusion

This research tried to decode the linguistic and thematic landscape of over 20,000 Italian song lyrics, bridging the gap between raw textual data and deep semantic understanding. By applying a Text Mining pipeline, we navigated the morphological complexities of the Italian language, evaluated the structural trade-offs between traditional Machine Learning and Deep Learning architectures, and extracted meaningful cultural insights regarding the national music scene.

We established that traditional, heuristic NLP tools are insufficient for handling the grammatical intricacies of Italian lyrics, particularly verb conjugations. The transition to Stanza’s neural pipeline proved that sacrificing computational speed is a necessary trade-off to achieve the morphological precision required for robust downstream modeling.

While traditional linear algorithms like NMF effectively captured broad dominant themes, the implementation of a Neural Variational Autoencoder (VAE) demonstrated vastly superior semantic depth. The VAE successfully isolated complex cultural tropes, such as modern urban slang, the classic Italian summer hit, and conscious music, proving the value of non-linear embeddings in capturing human narrative.

The multi-label genre classification task highlighted a fascinating architectural divergence. The traditional SVM One-vs-Rest model secured the best overall Macro F1-Score due to its mathematical resilience on highly complex, minority genres like Jazz and Electronic. Conversely, the fine-tuned UmBERTo Transformer dominated the most populated, mainstream genres (Cantautore, Classical, Pop,

Rock, and Hip-Hop). This indicates that while Transformers possess an unmatched contextual understanding of the Italian language, traditional SVMs remain highly valuable for maintaining baseline stability across imbalanced, long-tail distributions.

Bibliography

- Bianchi, F., Nozza, D., & Hovy, D. (2021,). "FEEL-IT: Emotion and Sentiment Classification for the Italian Language". *Proceedings of the 11th Workshop on Computational Approaches to Subjectivity, Sentiment and Social Media Analysis*.
- Blei, D. M., Ng, A. Y., & Jordan, M. I. (2003). Latent dirichlet allocation. *Journal of Machine Learning Research*, 3(Jan), 993–1022.
- Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794. <https://doi.org/10.1145/2939672.2939785>
- Deerwester, S., Dumais, S., Landauer, T., Furnas, G., & Harshman, R. (1990). Indexing by Latent Semantic Analysis. *Journal of the American Society for Information Science*, 41, 391–407. [https://doi.org/10.1002/\(sici\)1097-4571\(199009\)41:660;391::aid-asi162;3.0.co;2-9](https://doi.org/10.1002/(sici)1097-4571(199009)41:660;391::aid-asi162;3.0.co;2-9)
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, 4171–4186.
- Honnibal, M., Montani, I., Van Landeghem, S., & Boyd, A. (2020,). *spaCy: Industrial-strength Natural Language Processing in Python*. Zenodo. <https://doi.org/10.5281/zenodo.1212303>
- Lee, D. D., & Seung, H. S. (1999). Learning the parts of objects by non-negative matrix factorization. *Nature*, 401(6755), 788–791.
- Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., & Stoyanov, V. (2019). RoBERTa: A Robustly Optimized BERT Pretraining Approach. *Arxiv Preprint Arxiv:1907.11692*.
- Mimno, D., Wallach, H., Talley, E., Leenders, M., & McCallum, A. (2011). Optimizing semantic coherence in topic models. *Proceedings of the 2011 Conference on Empirical Methods in Natural Language Processing*, 262–272.
- Musixmatch Research. (2020,). *UmBERTo: An Italian Language Model trained with Whole Word Masking*. GitHub. <https://github.com/musixmatchresearch/umberto>
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., ... Chintala, S. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *Advances in Neural Information Processing Systems*, 32, 8024–8035. <https://proceedings.neurips.cc/paper/2019/file/bdbca288fee7f92f2bfa9f7012727740-Paper.pdf>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

- Qi, P., Zhang, Y., Zhang, Y., Bolton, J., & Manning, C. D. (2020). Stanza: A Python Natural Language Processing Toolkit for Many Human Languages. In A. Celikyilmaz & T.-H. Wen (Eds.), *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics: System Demonstrations: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*. <https://doi.org/10.18653/v1/2020.acl-demos.14>
- Salton, G., Wong, A., & Yang, C. S. (1975). A Vector Space Model for Automatic Indexing. *Communications of the ACM*, 18(11), 613–620. <https://doi.org/10.1145/361219.361220>
- Srivastava, A., & Sutton, C. (2017). Autoencoding variational inference for topic models. *Arxiv Preprint Arxiv:1703.01488*.
- Stahl, P. M. (2021,). *Lingua: The most accurate natural language detection library for Python*. GitHub. <https://github.com/pemistahl/lingua-py>
- Varoquaux, G., Grisel, O., & others. (2009,). *Joblib: running Python functions as pipeline jobs*. <https://joblib.readthedocs.io/>
- Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., Platen, P. von, Ma, C., Jernite, Y., Plu, J., Xu, C., Le Scao, T., Gugger, S., ... Rush, A. M. (2020). Transformers: State-of-the-Art Natural Language Processing. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 38–45. <https://aclanthology.org/2020.emnlp-demos.6>
- Zipf, G. K. (1949). *Human Behavior and the Principle of Least Effort: An Introduction to Human Ecology*. Addison-Wesley.

*This project is dedicated to our friend Nicola,
who decided not to take this course with us.*